PRISMSPACE - INTELLIGENT MULTI-AGENT PLATFORM



Dept. of Computer Science & Engineering
College of Engineering, Karunagappally

Project Group 01

NOBIN SIJO	KNP23CS082
PRANAV P	KNP23CS085
PRASANTH P	KNP23CS086
SANKAR NARAYAN S	KNP23CS094

Project Coordinator & Guide
Dr. Leena Silvoster
Associate Professor
College of Engineering, Karunagappally

01. Introduction: PrismSpace Platform

PrismSpace is a local-first, multi-agent intelligence platform that converts natural-language user goals into dependency-aware **Directed Acyclic Graphs (DAGs)** for parallel sub-task execution. By pairing dynamic multi-model routing (Groq, OpenAI, NVIDIA NIM) with standardized tool access via the **Model Context Protocol (MCP)**, it coordinates specialized AI swarms to solve everyday and developer workflows—delivering safety-aligned verification and zero-latency local persistence (`PrismDB`) within a unified, reactive workspace.

Dynamic DAG Planning **MCP Tool Swarms** **Multi-Model Routing** **Local-First PrismDB**

02. Project Objectives

1. **Context-Aware Task Planning:** Convert everyday user goals (communication, file management, daily schedules) and technical requests into structured Directed Acyclic Graphs (DAGs) for organized, dependency-safe execution.

2. **Intelligent Model & Agent Routing:** Dynamically route each sub-task to the most suitable Large Language Model (LLM) backend and dispatch specialized, context-aware agents in parallel.

3. **Practical Tool Execution via MCP:** Connect agents to everyday productivity tools and development utilities (Gmail, Google Drive, Filesystem, Terminal CLI, GitHub) through the Model Context Protocol (MCP), accompanied by live Server-Sent Events (SSE) updates.

4. **Safety Verification & Local Persistence:** Validate outputs with human approval checkpoints and Odds Ratio Preference Optimization (ORPO), storing results reactively in local IndexedDB (Indexed Database: `PrismDB`).

03. PrismSpace Workflow (Figure 1.0)

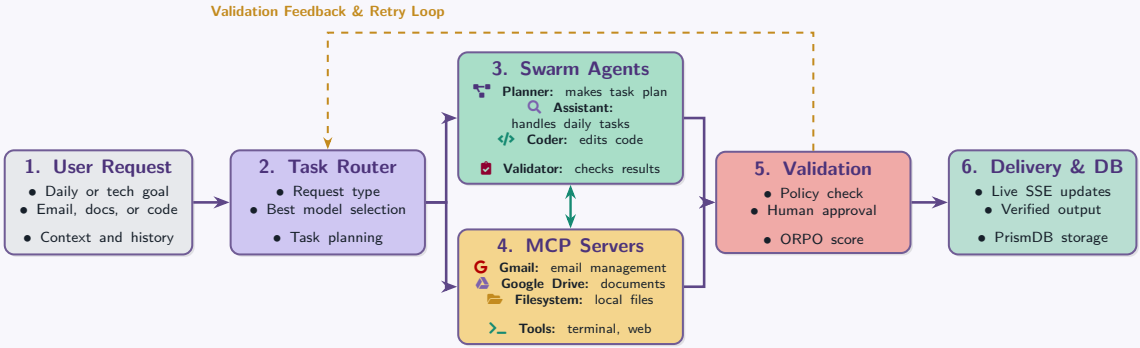
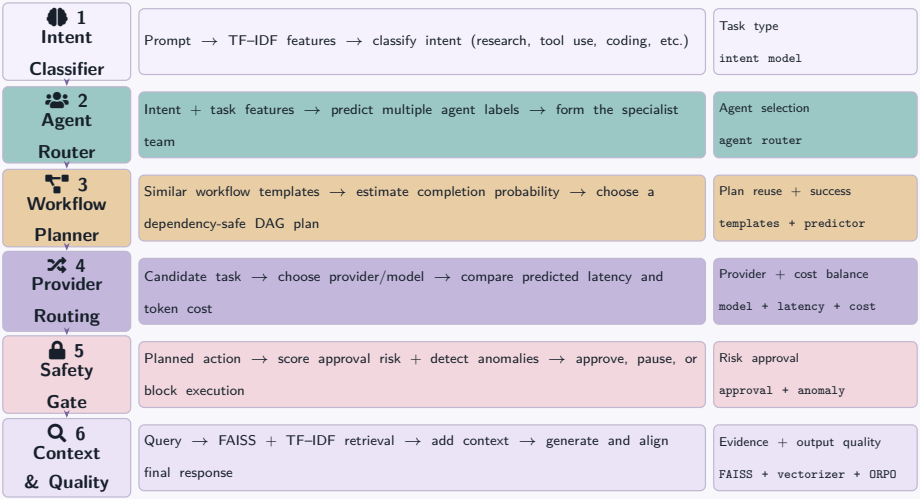


Figure 1.0: PrismSpace Workflow | **DAG** = Directed Acyclic Graph | **MCP** = Model Context Protocol | **ORPO** = Odds Ratio Preference Optimization | **SSE** = Server-Sent Events

04. System Design Principles

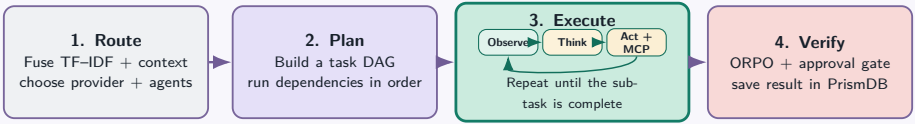
- **Layered Decoupling:** Separates the React client UI, FastAPI bridge, and foundation LLMs (each layer has one clear role).
- **Dynamic DAG Scheduling:** Runs dependent sub-tasks in parallel ($K \leq 5$; additional tasks are queued) for daily and technical workflows.
- **Standardized Protocol (MCP):** Uses open JSON-RPC tool contracts (a common way for agents and tools to communicate).
- **Human-in-the-Loop:** Sensitive operations require explicit user approval (a safety checkpoint before execution).
- **Local IndexedDB (`PrismDB`):** ACID browser storage (reliable local data) for sessions, tasks, tool history, and files.
- **Reactive Subscriptions:** Dexie `useLiveQuery()` updates the UI automatically when data changes.
- **Server-Sent Events (SSE):** Streams tokens and agent telemetry over persistent HTTP (live one-way updates).
- **Calibrated Preference Gating:** ORPO ranks and filters outputs to improve quality and safety.
- **Design Philosophy:** Context-aware, sandboxed, and observable multi-agent workspace (safe and traceable by design).

05. Model Responsibilities & Inference Flow



06. Proposed Methods

Goal: Convert a natural-language request into a planned, tool-assisted, and safety-checked result.



Main method: The DAG decides *what can run next*; the ReAct loop decides *how each agent acts* using MCP tools.

Supporting methods: multi-head routing selects the right specialist; ORPO and the approval gate filter risky or low-quality actions; PrismDB keeps the client state reactive.

08. How the System Works: From Request to Result

Stage 1: Ingestion & Features

- **Capture request:** Read the user's goal.
- **Clean text:** Remove noise and missing values.
- **Extract features:** Convert the request into useful signals.

Stage 2: Intelligence & DAG

- **Classify intent:** Decide what kind of task it is.
- **Choose agents and models:** Match each sub-task to the right worker.
- **Build the DAG:** Arrange tasks in dependency order and check safety.

Stage 3: Execution & Storage

- **Execute:** Run ready tasks with MCP tools.
- **Stream updates:** Show progress and logs live.
- **Check and save:** Verify the result and store the run.

How a DAG is created

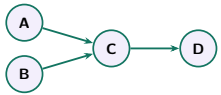
1. User request
"Plan a report"

→ **2. Split into sub-tasks** → **3. Add dependencies** → **4. Create DAG $\mathcal{G}(V, E)$** → **5. Run ready tasks in parallel**

Dependency-Aware Topological Execution:

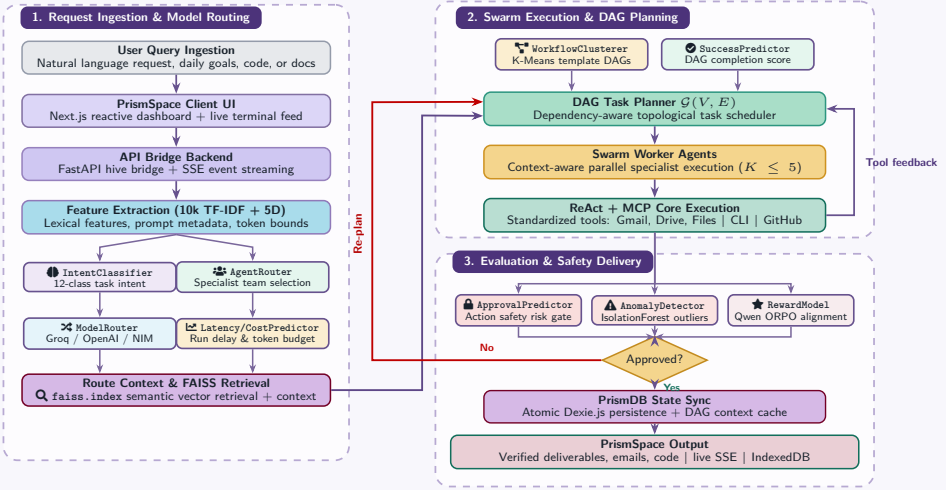
Independent sub-tasks execute concurrently ($A \parallel B$). Downstream tasks wait until prerequisite dependencies resolve before starting.

Example:



A/B run together → C waits for both → D finishes

09. PrismSpace System Architecture & Multi-Model Swarm Workflow (Figure 1.2)



10. Unified Dataset Preparation & Multi-Model Training Pipeline

1. Data Sources

24 training sets covering:

- Agent traces: timings, tool calls, and task trajectories.
- Tool calling: API, parallel-call, and coding benchmarks.
- Retrieval data: search corpora, schemas, and dialogues.
- Safety and preference data: harm labels and ranked responses.

2. Curate & Train

- `dataset_loader.py`: combines JSON, CSV, Parquet, PKL, and Markdown.
- Normalizes prompts, actions, timestamps, and labels into one schema.
- Supervised preparation creates approval, success, provider, and reward splits.
- `train.py`: uses TF-IDF/FAISS retrieval, K-Means workflow clustering, and scikit-learn predictors.
- ORPO fine-tunes the reward model on Qwen2.5 for preference alignment.

3. Artifacts & Evaluate

- **11 production artifacts** in `Models/`.
- Intent classifier and multi-label router classify tasks and select specialists.
- K-Means clusters workflows; predictors estimate success, latency, and cost.
- Approval classifier and Isolation Forest screen risky or unusual actions.
- TF-IDF/FAISS retrieve context; ORPO aligns responses.
- Holdout tests check F1, precision, recall, calibration, GAIA, and SWE-bench.



09. Browser Database Schema: PrismDB (Figure 1.3)



agent_chat_sessions

&id	string (UUID, PK)
title	string
updatedAt	number (Index)
createdAt	number (Index)

agent_chat_messages

++id	number (Auto-Inc PK)
sessionId	string (FK → sessions.id)
[sessionId+createdAt]	Compound Index
agentId	string (Index)
role	'user' | 'assistant' | 'tool'
status	'pending'|'streaming'|'done'
content	string (Text)
createdAt	number (Index)

ai_tool_history

++id	number (Auto-Inc PK)
sessionId	string (FK → sessions.id)
toolType	string (bash|web|code)
input	text (Args / JSON Payload)
output	text (Logs / Return Value)
createdAt	number (Index)

Agent Key-Value Stores

&key `settings`	JSON (API keys, model config)
&key `agent_state`	Active Swarm DAG
&key `user_profile`	JSON (System Instructions)
&key `files`	Blob / MimeType (Deliverables)

Figure 1.3: PrismDB — AI-Agent Browser IndexedDB Schema (Dexie.js v3) | **PK** = Primary Key **FK** = Foreign Key **Index** = B-tree index | FK / Indexed row Key-Value store

`sessions.id` shared FK | `sessions.id` → `messages.sessionId` | `sessions.id` → `tool_history.sessionId` | `useLiveQuery()` · ACID transactions · Dexie.js v3

Algorithm: Step 1 – Understand the User Prompt

Step 1: Prompt Understanding

1. Read the request and identify the user's main goal.
2. Extract constraints, files, preferences, deadlines, and expected results.
3. Add relevant conversation history and project context.
4. Detect whether the request needs tools, human approval, or extra clarification.
5. Create a structured task summary for the router.

Methods used: natural language understanding, context retrieval, intent and constraint extraction.

Algorithm: Step 2 – Classify the Request

Step 2: Intent and Requirement Analysis

1. Classify the request as an everyday, document, research, coding, or mixed task.
2. Identify required capabilities such as reading, writing, searching, or communication.
3. Estimate task complexity, urgency, and risk.
4. Mark actions that are read only, reversible, or sensitive.

Methods used: intent classification, keyword and context matching, rule based safety screening.

Algorithm: Step 3 – Plan and Route the Task

Step 3: Task Decomposition and Model Routing

1. Break the request into smaller dependent or independent tasks.
2. Build a Directed Acyclic Graph (DAG) showing task order and dependencies.
3. Select the best agent for each task: Planner, Assistant, Researcher, Coder, or Reviewer.
4. Route each task to a suitable model based on reasoning need, speed, and cost.
5. Run independent tasks in parallel when possible.

Key equation: $\text{Model}^* = \arg\min_m(\text{cost}_m + \text{latency}_m)$ subject to task quality. **Methods used:** DAG planning, topological ordering, multi provider model routing, parallel dispatch.

Algorithm: Step 4 – Select the Agent Team

Step 4: Swarm Agent Selection

1. The Planner Agent organizes the task and tracks dependencies.
2. The Assistant Agent handles email, documents, schedules, and daily tasks.
3. The Researcher Agent gathers and summarizes relevant information.
4. The Coder Agent writes, edits, and tests code.
5. The Reviewer Agent checks intermediate work and sends feedback for correction.

Methods used: role based agent dispatch, capability matching, concurrent swarm execution, shared context.

Algorithm: Step 5 – Execute with MCP Tools

Step 5: ReAct Reasoning and MCP Tool Use

1. The agent observes the current task state and decides the next action.
2. It calls the required MCP server: Gmail, Google Drive, Filesystem, Terminal, or Web.
3. The tool result is returned to the agent as new context.
4. The agent continues until the assigned subtask is complete.
5. Sensitive actions pause at the human approval checkpoint.

Core loop: Observation → Thought → Action → Observation. **Methods used:** ReAct loop, MCP JSON RPC tool calls, tool result feedback, human in the loop approval.

Algorithm: Step 6 – Share Context and Memory

Step 6: Context Synchronization

1. Pass completed subtask results to the next dependent agent.
2. Keep the original request, constraints, tool results, and decisions together.
3. Remove duplicate or outdated context before sending the next task.
4. Update the task state, agent status, and completed DAG nodes.
5. Save useful conversation and tool state locally in PrismDB.

Methods used: shared task memory, state synchronization, DAG node updates, Dexie.js and IndexedDB persistence.

Algorithm: Step 7 – Validate and Retry

Step 7: Verification and Safety

1. The Reviewer or Validator Agent checks correctness and completeness.
2. Check that the response follows the user's request and project constraints.
3. Check tool actions for unsafe commands, unauthorized access, or policy violations.
4. Use human approval for high impact actions.
5. If verification fails, return the task to the relevant agent with corrective feedback.

Key equation: Accept when quality score $\geq \tau_{\text{safety}}$. **Methods used:** rule based validation, completeness checking, human approval gates, ORPO preference scoring, retry loop.

Algorithm: Step 8 – Deliver and Monitor

Step 8: Verified Delivery and Monitoring

1. Combine verified subtask results into the final answer or deliverable.
2. Stream progress, tool events, and final results to the interface using SSE.
3. Store the final response, task state, and useful metadata in PrismDB.
4. Record agent latency, tool status, failures, and approval decisions.
5. Use telemetry to improve future routing and task execution.

Methods used: Server Sent Events, reactive UI updates, IndexedDB persistence, telemetry collection, post task review.

12. Conclusion & Key Contributions

Key Technical Contributions

- **Multi-Agent Support:** Handles daily tasks and coding.
- **Standardized Tools:** Uses MCP to connect agents and tools.
- **LLM Flexibility:** Supports different AI models for speed, quality, and cost.
- **Safe AI:** Uses approval checks and preference-based training.

Impact & System Advantages

- **Clear Monitoring:** Users can see agent actions and tool calls.
- **Fast Updates:** The interface responds with minimal delay.
- **Controlled Automation:** Users approve important actions.
- **Easy to Extend:** New agents and skills can be added easily.

Conclusion: PrismSpace is a flexible, observable, and safe multi-agent AI framework for everyday tasks and software development.

Thank You!

Questions & Discussion

Nobin Sijo | Pranav P | Prasanth P | Sankar Narayan S

Department of Computer Science & Engineering
College of Engineering, Karunagappally